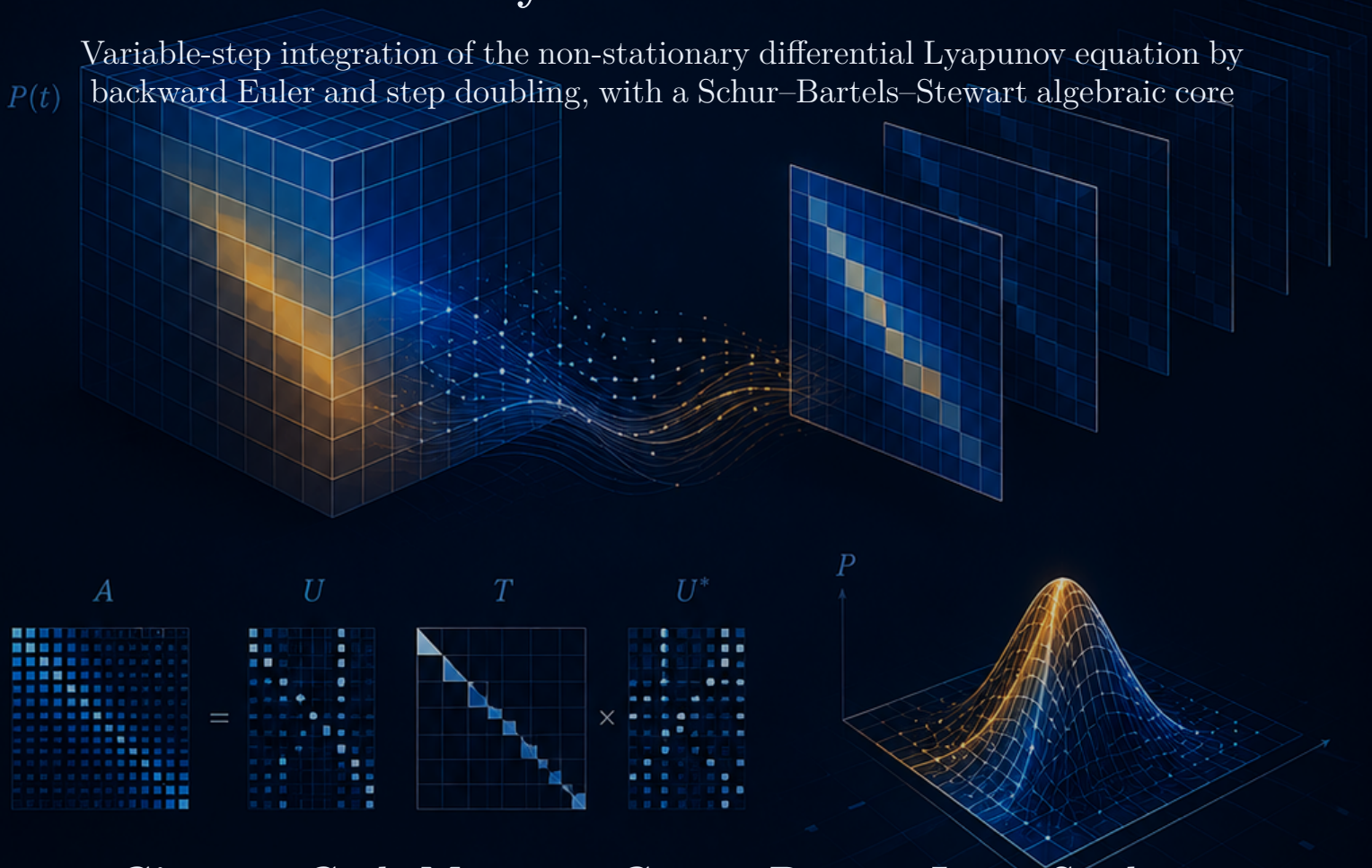


Backward-Euler Matrix Lyapunov eq. Solver

Theory and User Manual

Variable-step integration of the non-stationary differential Lyapunov equation by backward Euler and step doubling, with a Schur–Bartels–Stewart algebraic core



Giuseppe Carlo Marano • Gaurav Datta • Laura Sardone

Dept. of Structural, Geotechnical and Building Engineering (DISEG) • Politecnico di Torino, Turin, Italy

Research Prototype • Software v1.0.0 • Manual English ed. 1.0 • September 2026

Abstract

This manual documents a research MATLAB package for the time-domain covariance analysis of linear time-invariant and time-varying structural systems subjected to non-stationary stochastic excitation, including systems written in augmented state space with input-generating filters. The response second-moment statistics are obtained by integrating the non-stationary differential Lyapunov equation $\dot{\mathbf{R}} = \mathbf{A}(t)\mathbf{R} + \mathbf{R}\mathbf{A}^\top(t) + \mathbf{B}(t)$ directly in matrix form. Time integration uses a variable-step *backward-Euler* method in which every implicit update is recast as an algebraic Lyapunov equation and solved by a reusable complex-Schur Bartels–Stewart factorization; the solver is implemented in base MATLAB and does not require the Control System Toolbox. Temporal accuracy is controlled by *step doubling*: one full step is compared with two consecutive half steps, yielding a Richardson local-error estimate that drives a proportional or proportional–integral step-size controller through a physically scaled, weighted root-mean-square Frobenius norm. A distinguishing property of the scheme is that the accepted covariance is symmetric and, under a non-expansiveness condition on $\mathbf{A}(t)$ with $\mathbf{B}(t) \succeq \mathbf{0}$ and $\mathbf{R}_0 \succeq \mathbf{0}$, provably positive semidefinite; the first-order backward-Euler route is adopted precisely for this unconditional physical-admissibility guarantee, in deliberate contrast to higher-order alternatives. The package returns covariance, variance, interstorey-drift, velocity, and acceleration statistics, and is validated against an analytical scalar solution, the stationary algebraic Lyapunov limit, symmetry and positive-semidefiniteness checks, and tolerance-monotonicity tests, with single-degree-of-freedom and six-storey shear-building examples. First- and second-order covariance sensitivities and second-moment uncertainty propagation are developed analytically in an appendix as a planned extension; they are documented for completeness and are not part of the present numerical core.

Keywords: non-stationary random vibration; differential Lyapunov equation; covariance analysis; backward Euler; adaptive time stepping; step doubling; complex-Schur Bartels–Stewart solver; positive semidefiniteness; MATLAB.

Document control

Document	Backward-Euler Matrix Lyapunov eq. Solver — Theory and User Manual
Version	English edition 1.0 (software v1.0.0)
Language	English
Repository	https://github.com/ArtISte-Research-Group/adaptive-lyapunov-solver
Primary entry points	<code>lyap_be_adaptive</code> , <code>lyap_be_options</code> , <code>lyap_schur_factor</code> / <code>lyap_schur_solve</code>
Numerical core	Variable-step backward Euler with step doubling; complex-Schur Bartels–Stewart algebraic Lyapunov solve
Error control	Richardson local-error estimate; physically scaled weighted-RMS Frobenius norm; P / PI step-size controller
Stability	A- and L-stable; provable positive-semidefiniteness preservation under a non-expansiveness condition
Dependencies	Base MATLAB; no Control System Toolbox required

Contents

1	Purpose and scope	4
2	Stochastic state-space formulation	4
2.1	Linear stochastic system	4
2.2	Derivation of the differential Lyapunov equation	5
2.3	Transition-matrix representation	5
2.4	Stationary limit	5
3	Backward-Euler discretization	5
3.1	Variable time grid	5
3.2	Conversion to an algebraic Lyapunov equation	6
3.3	Vectorized form and why it is avoided	6
3.4	Existence and uniqueness	6
4	Adaptive step size by step doubling	6
4.1	Full step and two half steps	7
4.2	Local error estimate	7
4.3	Physically scaled error norm	8
4.4	Proportional controller	8
4.5	PI controller	8
4.6	Rejected-step update	9
5	Flow diagrams	10
6	Algebraic Lyapunov solver	11
6.1	Complex Schur factorization	11
6.2	Columnwise triangular solution	11
6.3	Residual	11
6.4	Factorization reuse	12
6.5	Computational complexity	12
7	Symmetry and positive semidefiniteness	12
8	Software architecture	13
9	Installation and first run	13
9.1	Requirements	13
9.2	Quick start	14

10 Main interface	14
10.1 Input arguments	14
10.2 Options	14
10.3 Output structure	15
11 Example 1: modulated SDOF oscillator	15
11.1 Mechanical model	15
11.2 Component covariance equations	16
11.3 MATLAB example	16
12 Example 2: time-varying state matrix	17
13 Example 3: shear-building model	18
13.1 Equations of motion	18
13.2 Interstorey drift covariance	18
13.3 Acceleration covariance	19
14 Input filters and augmented state space	19
15 Tolerance selection and numerical practice	19
15.1 Recommended starting values	19
15.2 Selecting StateScale	20
16 Validation and included tests	20
16.1 Exact scalar solution	20
16.2 Recommended independent checks	21
17 Extension to first- and second-order sensitivities	21
17.1 Backward-Euler sensitivity updates	22
17.2 Dimensionless error control for sensitivities	22
17.3 Output functionals	22
18 Second-order uncertainty propagation	23
19 Method limitations	23
20 Troubleshooting	23
A Complete pseudocode	24
B Output callback	25
C Reproducibility checklist	25

1 Purpose and scope

The package integrates the non-stationary differential Lyapunov equation

$$\dot{\mathbf{R}}(t) = \mathbf{A}(t)\mathbf{R}(t) + \mathbf{R}(t)\mathbf{A}^\top(t) + \mathbf{B}(t), \quad \mathbf{R}(t_0) = \mathbf{R}_0, \quad (1)$$

by a variable-step backward-Euler method. Temporal error is controlled through *step doubling*: one full step is compared with two consecutive half steps. Every implicit update is retained in matrix form and converted into an algebraic Lyapunov equation.

The package is intended for:

- covariance analysis of linear systems subjected to non-stationary stochastic excitation;
- structural systems written in augmented state space, including input-generating filters;
- transient variance, covariance, drift, velocity, and acceleration evaluation;
- convergence and work-precision studies;
- research prototypes for first- and second-order design sensitivities.

The implementation does not call MATLAB's `lyap` function. It includes a complex-Schur Bartels–Stewart solver, and therefore does not require the Control System Toolbox. The matrix-oriented formulation follows the classical theory of Sylvester and Lyapunov equations [1] and the structural covariance literature [2, 3]. Direct numerical integration of the non-stationary covariance equation along these lines has been developed for linear multi-degree-of-freedom systems [4], extended to nonlinear systems through equivalent stochastic linearization [5, 6], and applied to the seismic covariance analysis of multi-storey buildings [7]. Alternative treatments of non-stationary response include evolutionary frequency-response and spectral-moment formulations [8, 9] and transient probability-density approaches [10].

2 Stochastic state-space formulation

2.1 Linear stochastic system

The covariance formulation summarised here is standard in random-vibration and structural-dynamics theory [11, 12]. A convenient rigorous formulation uses an Itô stochastic differential equation:

$$d\mathbf{z}(t) = \mathbf{A}(t)\mathbf{z}(t)dt + \mathbf{G}(t)d\mathbf{W}(t), \quad (2)$$

where $\mathbf{z}(t) \in \mathbb{R}^m$, $\mathbf{W}(t)$ is a vector Wiener process, and

$$\mathbb{E}[d\mathbf{W}(t)d\mathbf{W}^\top(t)] = \mathbf{J}(t)dt. \quad (3)$$

The same model may be written formally as

$$\dot{\mathbf{z}}(t) = \mathbf{A}(t)\mathbf{z}(t) + \mathbf{G}(t)\mathbf{w}(t), \quad (4)$$

where $\mathbf{w}(t)$ is white noise with instantaneous intensity $\mathbf{J}(t)$.

For a nonzero mean, define

$$\boldsymbol{\mu}_z(t) = \mathbb{E}[\mathbf{z}(t)], \quad (5)$$

which satisfies

$$\dot{\boldsymbol{\mu}}_z(t) = \mathbf{A}(t)\boldsymbol{\mu}_z(t). \quad (6)$$

The centered covariance is

$$\mathbf{R}(t) = \mathbb{E}[(\mathbf{z} - \boldsymbol{\mu}_z)(\mathbf{z} - \boldsymbol{\mu}_z)^\top]. \quad (7)$$

For zero mean, this reduces to $\mathbf{R} = \mathbb{E}[\mathbf{z}\mathbf{z}^\top]$.

2.2 Derivation of the differential Lyapunov equation

Applying Itô's product rule,

$$d(\mathbf{z}\mathbf{z}^\top) = (d\mathbf{z})\mathbf{z}^\top + \mathbf{z}(d\mathbf{z})^\top + (d\mathbf{z})(d\mathbf{z})^\top. \quad (8)$$

Using eqs. (2) and (3) and taking expectations gives

$$\dot{\mathbf{R}}(t) = \mathbf{A}(t)\mathbf{R}(t) + \mathbf{R}(t)\mathbf{A}^\top(t) + \mathbf{B}(t), \quad (9)$$

with

$$\boxed{\mathbf{B}(t) = \mathbf{G}(t)\mathbf{J}(t)\mathbf{G}^\top(t) \succeq \mathbf{0}.} \quad (10)$$

The matrix $\mathbf{B}(t)$ is the instantaneous covariance injection rate.

2.3 Transition-matrix representation

Let $\Phi(t, \tau)$ be the state-transition matrix defined by

$$\frac{\partial}{\partial t}\Phi(t, \tau) = \mathbf{A}(t)\Phi(t, \tau), \quad \Phi(\tau, \tau) = \mathbf{I}. \quad (11)$$

The covariance solution is

$$\boxed{\mathbf{R}(t) = \Phi(t, t_0)\mathbf{R}_0\Phi^\top(t, t_0) + \int_{t_0}^t \Phi(t, \tau)\mathbf{B}(\tau)\Phi^\top(t, \tau) d\tau.} \quad (12)$$

This representation proves that $\mathbf{R}(t) \succeq \mathbf{0}$ whenever $\mathbf{R}_0 \succeq \mathbf{0}$ and $\mathbf{B}(t) \succeq \mathbf{0}$.

2.4 Stationary limit

If $\mathbf{A}$ and $\mathbf{B}$ are constant and $\mathbf{A}$ is Hurwitz,

$$\lim_{t \rightarrow \infty} \mathbf{R}(t) = \overline{\mathbf{R}}, \quad (13)$$

where $\overline{\mathbf{R}}$ is the unique solution of

$$\boxed{\mathbf{A}\overline{\mathbf{R}} + \overline{\mathbf{R}}\mathbf{A}^\top + \mathbf{B} = \mathbf{0}.} \quad (14)$$

For constant $\mathbf{A}$, the exact transient formula is

$$\mathbf{R}(t) = e^{\mathbf{A}(t-t_0)}\mathbf{R}_0e^{\mathbf{A}^\top(t-t_0)} + \int_{t_0}^t e^{\mathbf{A}(t-\tau)}\mathbf{B}(\tau)e^{\mathbf{A}^\top(t-\tau)} d\tau. \quad (15)$$

Convolution integrals of this type can be evaluated directly through block matrix-exponential techniques [13], which provide an independent reference solution for the constant-coefficient validation cases.

3 Backward-Euler discretization

3.1 Variable time grid

Let

$$t_{n+1} = t_n + h_n, \quad h_n > 0. \quad (16)$$

Backward Euler applied to eq. (1) gives

$$\frac{\mathbf{R}_{n+1} - \mathbf{R}_n}{h_n} = \mathbf{A}_{n+1}\mathbf{R}_{n+1} + \mathbf{R}_{n+1}\mathbf{A}_{n+1}^\top + \mathbf{B}_{n+1}, \quad (17)$$

where

$$\mathbf{A}_{n+1} = \mathbf{A}(t_{n+1}), \quad \mathbf{B}_{n+1} = \mathbf{B}(t_{n+1}). \quad (18)$$

Rearranging,

$$\mathbf{R}_{n+1} - h_n\mathbf{A}_{n+1}\mathbf{R}_{n+1} - h_n\mathbf{R}_{n+1}\mathbf{A}_{n+1}^\top = \mathbf{R}_n + h_n\mathbf{B}_{n+1}. \quad (19)$$

3.2 Conversion to an algebraic Lyapunov equation

Define the shifted matrix and right-hand side

$$\mathbf{M}_{n+1} = \mathbf{A}_{n+1} - \frac{1}{2h_n}\mathbf{I}, \quad \mathbf{Q}_{n+1} = \mathbf{B}_{n+1} + \frac{1}{h_n}\mathbf{R}_n. \quad (20)$$

Then eq. (19) is exactly equivalent to

$$\boxed{\mathbf{M}_{n+1}\mathbf{R}_{n+1} + \mathbf{R}_{n+1}\mathbf{M}_{n+1}^\top + \mathbf{Q}_{n+1} = \mathbf{0}.} \quad (21)$$

Thus, one backward-Euler substep is one algebraic Lyapunov solve.

The equivalent alternative shift

$$\widehat{\mathbf{M}}_{n+1} = \frac{h_n}{2}\mathbf{A}_{n+1} - \frac{1}{2}\mathbf{I} \quad (22)$$

leads to the same solution after multiplying the entire equation by h_n . The implementation uses eq. (20).

3.3 Vectorized form and why it is avoided

Using $\text{vec}(\mathbf{AXB}) = (\mathbf{B}^\top \otimes \mathbf{A})\text{vec}(\mathbf{X})$, eq. (17) could be written as

$$[\mathbf{I}_{m^2} - h_n(\mathbf{I}_m \otimes \mathbf{A}_{n+1} + \mathbf{A}_{n+1} \otimes \mathbf{I}_m)]\text{vec}(\mathbf{R}_{n+1}) = \text{vec}(\mathbf{R}_n + h_n\mathbf{B}_{n+1}). \quad (23)$$

A dense factorization of the $m^2 \times m^2$ coefficient matrix has an $O(m^6)$ trend. The matrix formulation instead uses an $m \times m$ Schur factorization and has an $O(m^3)$ trend.

3.4 Existence and uniqueness

The algebraic equation

$$\mathbf{MX} + \mathbf{XM}^\top + \mathbf{Q} = \mathbf{0} \quad (24)$$

has a unique solution if

$$\lambda_i(\mathbf{M}) + \overline{\lambda_j(\mathbf{M})} \neq 0 \quad \text{for all } i, j. \quad (25)$$

For stable systems, the shift $-\mathbf{I}/(2h_n)$ moves the spectrum further into the left half-plane. Backward Euler is A-stable and L-stable, which makes it robust for stiff systems, although its global order is only one [14].

4 Adaptive step size by step doubling

Variable step-size strategies for stiff Lyapunov differential equations have been studied previously [15, 16]; the present method follows the same aim but uses step doubling and a physically scaled error norm for its local-error estimate.

4.1 Full step and two half steps

For a trial step h , define the backward-Euler map

$$\mathcal{E}_h(t, \mathbf{R}) = \text{solution at } t + h \text{ of eq. (21)}. \quad (26)$$

The code computes

$$\mathbf{R}_{n+1}^{[h]} = \mathcal{E}_h(t_n, \mathbf{R}_n), \quad (27)$$

$$\mathbf{R}_{n+1/2}^{[h/2]} = \mathcal{E}_{h/2}(t_n, \mathbf{R}_n), \quad (28)$$

$$\mathbf{R}_{n+1}^{[h/2, h/2]} = \mathcal{E}_{h/2}\left(t_n + \frac{h}{2}, \mathbf{R}_{n+1/2}^{[h/2]}\right). \quad (29)$$

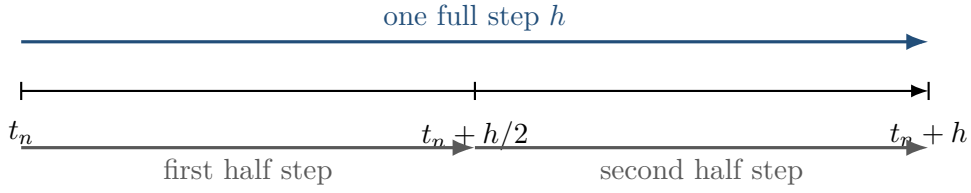


Figure 1: Temporal layout of step doubling.

4.2 Local error estimate

For a method of global order p , the local error over one interval is $O(h^{p+1})$. Let $\mathbf{R}_{n+1}^*$ denote the exact solution. To leading order,

$$\mathbf{R}_{n+1}^{[h]} = \mathbf{R}_{n+1}^* + C_e h^{p+1} + O(h^{p+2}), \quad (30)$$

$$\mathbf{R}_{n+1}^{[h/2, h/2]} = \mathbf{R}_{n+1}^* + \frac{C_e}{2^p} h^{p+1} + O(h^{p+2}). \quad (31)$$

Therefore, the error of the refined solution is estimated by

$$\mathbf{E}_{n+1}^{\text{est}} = \frac{\mathbf{R}_{n+1}^{[h/2, h/2]} - \mathbf{R}_{n+1}^{[h]}}{2^p - 1}. \quad (32)$$

This is the classical Richardson step-doubling estimate [14], applied here elementwise to the covariance matrix rather than to a state vector. For backward Euler, $p = 1$, so

$$\boxed{\mathbf{E}_{n+1} = \mathbf{R}_{n+1}^{[h/2, h/2]} - \mathbf{R}_{n+1}^{[h]}.} \quad (33)$$

The accepted state is

$$\boxed{\mathbf{R}_{n+1} = \mathbf{R}_{n+1}^{[h/2, h/2]}.} \quad (34)$$

The Richardson-extrapolated value

$$\mathbf{R}_{n+1}^{\text{Rich}} = \mathbf{R}_{n+1}^{[h/2, h/2]} + \mathbf{E}_{n+1} = 2\mathbf{R}_{n+1}^{[h/2, h/2]} - \mathbf{R}_{n+1}^{[h]} \quad (35)$$

is formally second order, but it is not used by default because it may lose positive semidefiniteness.

4.3 Physically scaled error norm

State components may have different units and magnitudes. Let

$$\mathbf{s} = [s_1, \dots, s_m]^\top, \quad s_i > 0, \quad (36)$$

and define

$$\mathbf{S} = \mathbf{s} \mathbf{s}^\top. \quad (37)$$

The dimensionless covariance, error, and old covariance are

$$\tilde{\mathbf{R}} = \mathbf{R} \oslash \mathbf{S}, \quad \tilde{\mathbf{E}} = \mathbf{E} \oslash \mathbf{S}, \quad \tilde{\mathbf{R}}_{\text{old}} = \mathbf{R}_{\text{old}} \oslash \mathbf{S}, \quad (38)$$

where $\oslash$ denotes elementwise division. The componentwise tolerance matrix is

$$\mathbf{W} = \text{atol} \mathbf{1} \mathbf{1}^\top + \text{rtol} \max(|\tilde{\mathbf{R}}_{\text{high}}|, |\tilde{\mathbf{R}}_{\text{old}}|), \quad (39)$$

where the maximum is elementwise. The code uses the weighted root-mean-square Frobenius norm

$$\text{err} = \frac{1}{\sqrt{m^2}} \|\tilde{\mathbf{E}} \oslash \mathbf{W}\|_F = \frac{1}{m} \|\tilde{\mathbf{E}} \oslash \mathbf{W}\|_F. \quad (40)$$

The step is accepted when

$$\boxed{\text{err} \leq 1}. \quad (41)$$

4.4 Proportional controller

Because the local error is $O(h^2)$, a proportional controller is

$$\boxed{f_{n+1} = \theta \text{err}_{n+1}^{-1/2}, \quad h_{n+1} = h_n f_{n+1}}, \quad (42)$$

where $0 < \theta < 1$ is a safety factor. In the implementation,

$$f_{n+1} \leftarrow \min(f_{\text{max}}, \max(f_{\text{min}}, f_{n+1})), \quad (43)$$

and

$$h_{n+1} \leftarrow \min(h_{\text{max}}, \max(h_{\text{min}}, h_n f_{n+1})). \quad (44)$$

4.5 PI controller

The default proportional-integral controller is

$$\boxed{f_{n+1} = \theta \text{err}_{n+1}^{-\alpha} \text{err}_n^\beta, \quad h_{n+1} = h_n f_{n+1}}. \quad (45)$$

The default exponents follow the standard proportional–integral step-size control values [14],

$$\alpha = \frac{0.7}{p+1} = 0.35, \quad \beta = \frac{0.4}{p+1} = 0.20, \quad p = 1, \quad (46)$$

so that

$$\boxed{h_{n+1} = \theta h_n \text{err}_{n+1}^{-0.35} \text{err}_n^{0.20}}. \quad (47)$$

4.6 Rejected-step update

If $\text{err} > 1$, the time and solution are not advanced. The retry factor is

$$f_{\text{rej}} = \theta \text{err}^{-1/2}, \quad (48)$$

then clipped as

$$f_{\text{rej}} \leftarrow \min(f_{\text{rej,max}}, \max(f_{\text{min}}, f_{\text{rej}})), \quad f_{\text{rej,max}} < 1. \quad (49)$$

The next attempt uses

$$\boxed{h_{\text{retry}} = \max(h_{\text{min}}, h_n f_{\text{rej}})}. \quad (50)$$

Default controller values are

$$\theta = 0.90, \quad f_{\text{min}} = 0.20, \quad f_{\text{max}} = 4.00, \quad f_{\text{rej,max}} = 0.80. \quad (51)$$

5 Flow diagrams

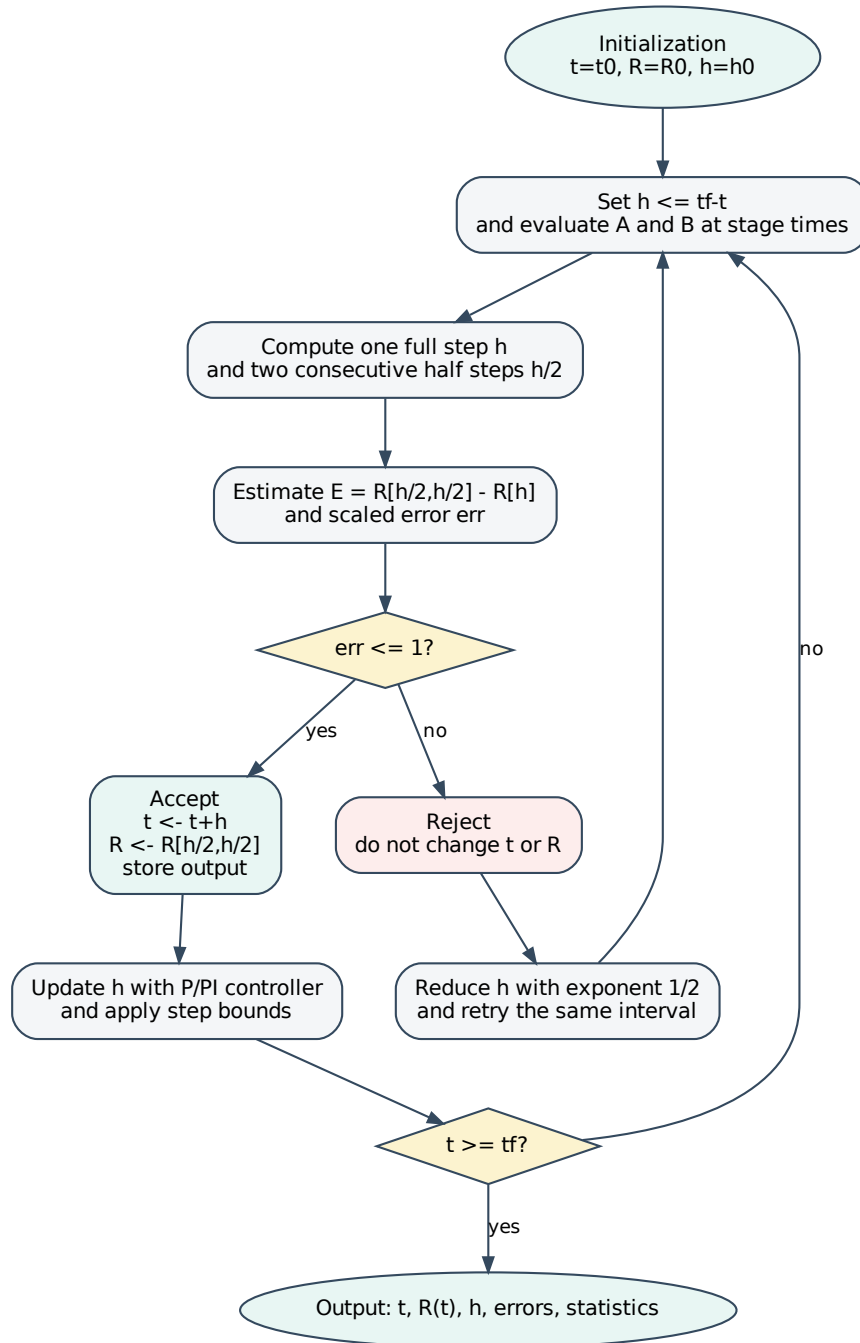


Figure 2: Complete adaptive marching procedure with step acceptance and rejection.



Figure 3: One backward-Euler Lyapunov substep.

6 Algebraic Lyapunov solver

6.1 Complex Schur factorization

For one substep, solve

$$MX + XM^T + Q = 0. \quad (52)$$

The complex Schur factorization is

$$M = UTU^*, \quad (53)$$

where U is unitary, T is upper triangular, and $(\cdot)^*$ denotes conjugate transpose. Set

$$X = UYU^*, \quad \hat{Q} = U^*QU. \quad (54)$$

Then

$$TY + YT^* + \hat{Q} = 0. \quad (55)$$

6.2 Columnwise triangular solution

Let y_j and $\hat{q}_j$ denote column j of Y and $\hat{Q}$. Since T is upper triangular, columns are computed for $j = m, m-1, \dots, 1$:

$$\left(T + \overline{T_{jj}}I\right) y_j = -\hat{q}_j - \sum_{k=j+1}^m y_k \overline{T_{jk}}. \quad (56)$$

After all columns have been obtained,

$$X = UYU^*. \quad (57)$$

For real input data, the real part is retained and symmetry is restored:

$$X \leftarrow \frac{X + X^T}{2}. \quad (58)$$

This is a complex-Schur implementation of the Bartels–Stewart method [1].

6.3 Residual

The optional relative algebraic residual is

$$\rho_{\text{Lyap}} = \frac{\|MX + XM^T + Q\|_F}{\max(1, \|Q\|_F)}. \quad (59)$$

The option `ResidualTolerance` is applied to this quantity.

6.4 Factorization reuse

For constant $\mathbf{A}$, a trial step h needs only two shifted matrices:

$$\mathbf{M}_h = \mathbf{A} - \frac{1}{2h}\mathbf{I}, \quad (60)$$

$$\mathbf{M}_{h/2} = \mathbf{A} - \frac{1}{h}\mathbf{I}. \quad (61)$$

The second factorization is reused for both half steps. If the controller retains the same h , both factorizations are reused on the next accepted interval.

For time-varying $\mathbf{A}(t)$, the three matrices are generally

$$\mathbf{M}_h = \mathbf{A}(t_n + h) - \frac{1}{2h}\mathbf{I}, \quad (62)$$

$$\mathbf{M}_{h/2}^{(1)} = \mathbf{A}(t_n + h/2) - \frac{1}{h}\mathbf{I}, \quad (63)$$

$$\mathbf{M}_{h/2}^{(2)} = \mathbf{A}(t_n + h) - \frac{1}{h}\mathbf{I}. \quad (64)$$

Thus, three factorizations are generally required for each attempted step.

6.5 Computational complexity

For dense matrices of order m :

$$\text{Schur factorization cost} = O(m^3), \quad (65)$$

$$\text{triangular Lyapunov solve cost} = O(m^3), \quad (66)$$

$$\text{current covariance storage} = O(m^2), \quad (67)$$

$$\text{full history storage} = O(N_t m^2). \quad (68)$$

Step doubling requires three Lyapunov solves per attempt. This cost is compensated by automatic concentration of time steps in rapidly varying phases.

7 Symmetry and positive semidefiniteness

If

$$\mathbf{R}_n \succeq \mathbf{0}, \quad \mathbf{B}_{n+1} \succeq \mathbf{0}, \quad (69)$$

then

$$\mathbf{Q}_{n+1} = \mathbf{B}_{n+1} + \frac{1}{h_n}\mathbf{R}_n \succeq \mathbf{0}. \quad (70)$$

If $\mathbf{M}_{n+1}$ is Hurwitz, the algebraic solution has the integral representation

$$\boxed{\mathbf{R}_{n+1} = \int_0^\infty e^{\mathbf{M}_{n+1}\tau} \mathbf{Q}_{n+1} e^{\mathbf{M}_{n+1}^\top \tau} d\tau \succeq \mathbf{0}.} \quad (71)$$

The numerical solution is symmetrized with eq. (58).

If `CheckPSD=true`, the code evaluates

$$\lambda_{\min}(\mathbf{R}_{n+1}). \quad (72)$$

A significant violation is measured relative to

$$\lambda_{\min}(\mathbf{R}_{n+1}) < -\varepsilon_{\text{PSD}} \max(1, \|\mathbf{R}_{n+1}\|_2). \quad (73)$$

If `ProjectPSD=true`, the eigendecomposition

$$\mathbf{R} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T \quad (74)$$

is replaced by

$$\mathbf{R}_+ = \mathbf{V} \max(\mathbf{\Lambda}, \mathbf{0}) \mathbf{V}^T. \quad (75)$$

Projection is not enabled by default because it modifies the numerical solution.

8 Software architecture

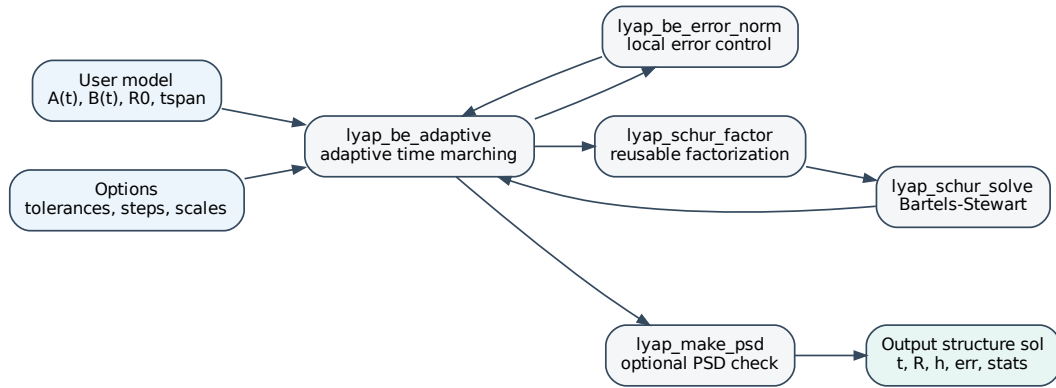


Figure 4: Relationship between user data, adaptive integration, Lyapunov routines, and output.

File	Purpose
lyap_be_adaptive.m	Time loop, step doubling, acceptance/rejection, factorization cache, and output structure.
lyap_be_options.m	Default options, name-value parsing, and validation.
lyap_be_error_norm.m	Weighted RMS Frobenius error norm in scaled covariance coordinates.
lyap_schur_factor.m	Complex Schur factorization of a shifted Lyapunov matrix.
lyap_schur_solve.m	Bartels–Stewart solution for one right-hand side.
lyap_make_psd.m	Optional minimum-eigenvalue check and PSD projection.
lyap_eval_matrix.m	Uniform evaluation of a constant matrix or a time-dependent function handle.

9 Installation and first run

9.1 Requirements

- MATLAB with the built-in `schur` function;
- no mandatory toolbox;
- sufficient memory for $m \times m \times N_t$ output if the full history is stored.

9.2 Quick start

Extract the package, set it as the current folder, and run

```
1 cd Adaptive_BE_Lyapunov_MATLAB
2 startup
3 run_tests
4 example_01_sdof_modulated
```

The command `startup` adds `src`, `examples`, and `tests` to the MATLAB path. The command `run_tests` executes the validation suite, and `example_01_sdof_modulated` runs the first worked example.

10 Main interface

The principal call is

```
1 sol = lyap_be_adaptive(Ainput, Binput, tspan, R0, options);
```

10.1 Input arguments

Argument	Meaning
<code>Ainput</code>	Constant state matrix $\mathbf{A}$, or function handle $@(t)$ $\mathbf{A}(t)$.
<code>Binput</code>	Constant input covariance rate $\mathbf{B}$, or function handle $@(t)$ $\mathbf{B}(t)$.
<code>tspan</code>	Two-entry vector $[t_0 \ t_f]$.
<code>R0</code>	Initial symmetric covariance matrix.
<code>options</code>	Structure returned by <code>lyap_be_options</code> .

10.2 Options

Option	Default	Meaning
<code>RelTol</code>	10^{-5}	Relative tolerance in scaled covariance coordinates.
<code>AbsTol</code>	10^{-9}	Absolute tolerance in scaled covariance coordinates.
<code>InitialStep</code>	automatic	First attempted step.
<code>MinStep</code>	automatic	Minimum admissible step.
<code>MaxStep</code>	span/10	Maximum admissible step.
<code>Safety</code>	0.90	Controller safety factor θ .
<code>MinFactor</code>	0.20	Minimum multiplicative step factor.
<code>MaxFactor</code>	4.00	Maximum multiplicative step factor.
<code>RejectMaxFactor</code>	0.80	Maximum factor after rejection.
<code>Controller</code>	'PI'	Either 'P' or 'PI'.
<code>PIAlpha</code>	0.35	Current-error exponent in the PI controller.
<code>PIBeta</code>	0.20	Previous-error exponent in the PI controller.
<code>StateScale</code>	ones	Positive state scale vector $\mathbf{s}$.
<code>MaxSteps</code>	100000	Maximum number of attempted steps.
<code>SymmetrizeInput</code>	true	Symmetrize $\mathbf{B}$ and $\mathbf{R}_0$.
<code>SymmetrizeSolution</code>	true	Symmetrize trial solutions.

Option	Default	Meaning
CheckPSD	true	Compute minimum covariance eigenvalues.
ProjectPSD	false	Clip negative covariance eigenvalues.
PSDTolerance	10^{-10}	Relative PSD diagnostic tolerance.
ComputeResidual	false	Evaluate algebraic Lyapunov residuals.
ResidualTolerance	10^{-9}	Warning threshold for eq. (59).
Verbose	false	Print accepted and rejected steps.
OutputFcn	empty	Callback <code>stop=f(t,R,flag)</code> .

10.3 Output structure

Field	Content
<code>sol.t</code>	Times of accepted steps.
<code>sol.R(:, :, k)</code>	Covariance matrix at <code>sol.t(k)</code> .
<code>sol.h</code>	Accepted step sizes.
<code>sol.err</code>	Accepted normalized local-error estimates.
<code>sol.minEigenvalue</code>	Minimum covariance eigenvalue at each output time, if enabled.
<code>sol.attempts</code>	Initial times, trial steps, errors, and acceptance flags for all attempts.
<code>sol.stats</code>	Accepted/rejected counts, Lyapunov solves, Schur factorizations, PSD projections, and maximum residual.
<code>sol.options</code>	Options actually used.

11 Example 1: modulated SDOF oscillator

11.1 Mechanical model

For the base-excited oscillator

$$m\ddot{x} + c\dot{x} + kx = -ma_g(t), \quad (76)$$

define

$$\omega = \sqrt{\frac{k}{m}}, \quad \zeta = \frac{c}{2m\omega}, \quad \mathbf{z} = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}. \quad (77)$$

The state matrix is

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -\omega^2 & -2\zeta\omega \end{bmatrix}. \quad (78)$$

For white-noise base acceleration

$$a_g(t) = \phi(t)w(t), \quad \mathbb{E}[w(t)w(\tau)] = 2\pi S_0\delta(t - \tau), \quad (79)$$

where $\phi(t)$ is a deterministic modulation function. Such envelope models originate in the early simulation literature [17–19]; energy-release-based definitions are also available [20]. The input vector and covariance rate are

$$\mathbf{g} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{B}(t) = 2\pi S_0\phi^2(t)\mathbf{g}\mathbf{g}^\top. \quad (80)$$

If the forcing is a physical force rather than base acceleration, use $\mathbf{g} = [0, 1/m]^\top$.

11.2 Component covariance equations

Let

$$\mathbf{R} = \begin{bmatrix} r_{xx} & r_{xv} \\ r_{xv} & r_{vv} \end{bmatrix}. \quad (81)$$

For $\mathbf{g} = [0, 1/m]^\top$, the differential Lyapunov equation becomes

$$\dot{r}_{xx} = 2r_{xv}, \quad (82)$$

$$\dot{r}_{xv} = r_{vv} - \omega^2 r_{xx} - 2\zeta\omega r_{xv}, \quad (83)$$

$$\dot{r}_{vv} = -2\omega^2 r_{xv} - 4\zeta\omega r_{vv} + \frac{2\pi S_0}{m^2} \phi^2(t). \quad (84)$$

The standard deviations are

$$\sigma_x(t) = \sqrt{r_{xx}(t)}, \quad \sigma_v(t) = \sqrt{r_{vv}(t)}. \quad (85)$$

11.3 MATLAB example

```

1 mass = 1.0;
2 omega = 2*pi*1.5;
3 zeta = 0.05;
4 A = [0 1; -omega^2 -2*zeta*omega];
5
6 g = [0; 1/mass];
7 S0 = 2e-2;
8 phi = @(t) jennings_envelope(t,2.0,12.0,0.30);
9 B = @(t) 2*pi*S0*phi(t).^2*(g*g');
10
11 opts = lyap_be_options( ...
12     'RelTol',1e-4, ...
13     'AbsTol',1e-6, ...
14     'InitialStep',0.01, ...
15     'MaxStep',0.50, ...
16     'StateScale',[0.10;1.00]);
17
18 sol = lyap_be_adaptive(A,B,[0 30],zeros(2),opts);
19
20 sigmaX = sqrt(max(0,squeeze(sol.R(1,1,:))));
21 sigmaV = sqrt(max(0,squeeze(sol.R(2,2,:))));

```

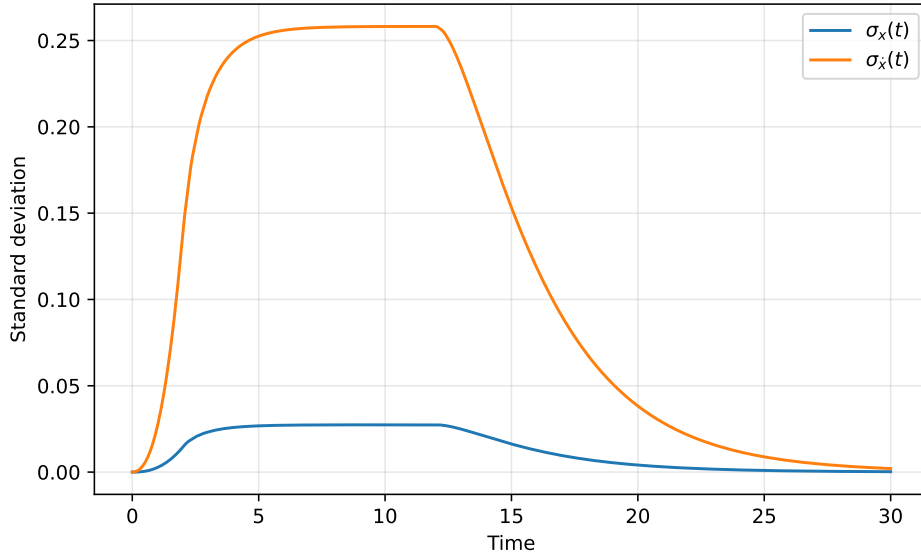


Figure 5: Evolving standard deviations for the modulated SDOF example.

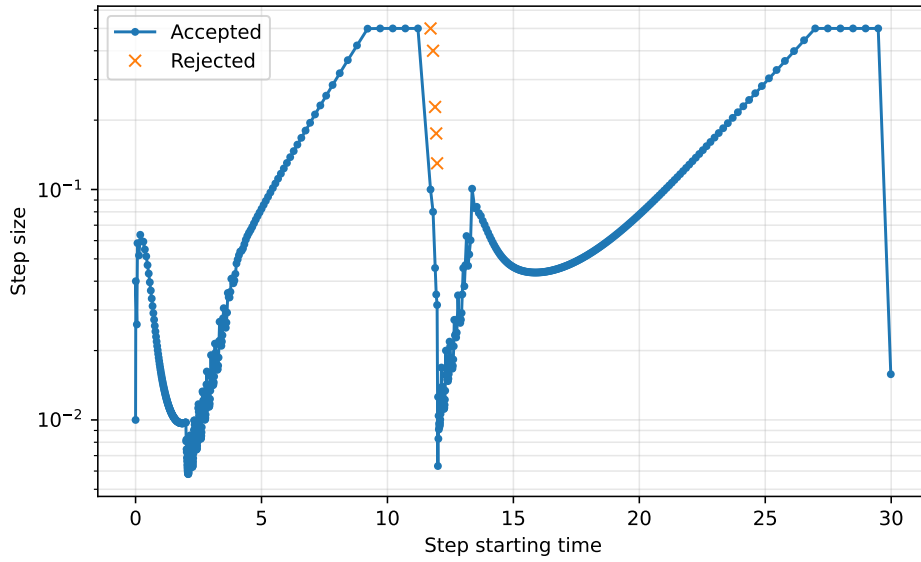


Figure 6: Accepted adaptive steps. The solver refines the build-up and transition phases and coarsens smooth tails.

12 Example 2: time-varying state matrix

A time-dependent natural frequency may be defined by

$$\omega(t) = \omega_0 [1 + 0.20 \sin(0.25t)], \quad (86)$$

with

$$\mathbf{A}(t) = \begin{bmatrix} 0 & 1 \\ -\omega^2(t) & -2\zeta\omega(t) \end{bmatrix}. \quad (87)$$

The MATLAB implementation is

```
1 omega0 = 2*pi*1.2;
```

```

2  zeta = 0.04;
3  A = @(t) [0 1; ...
4    -(omega0*(1+0.20*sin(0.25*t)))^2, ...
5    -2*zeta*omega0*(1+0.20*sin(0.25*t))];
6
7  sol = lyap_be_adaptive(A,B,[0 25],zeros(2),opts);

```

The code evaluates $\mathbf{A}(t)$ at each implicit endpoint. No frozen-operator approximation is introduced.

13 Example 3: shear-building model

13.1 Equations of motion

For an n -storey shear building [21],

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = -\mathbf{M}\boldsymbol{\tau}a_g(t), \quad (88)$$

where $\boldsymbol{\tau} = [1, \dots, 1]^\top$. With

$$\mathbf{z} = \begin{bmatrix} \mathbf{u} \\ \dot{\mathbf{u}} \end{bmatrix}, \quad (89)$$

the state matrix is

$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{M}^{-1}\mathbf{K} & -\mathbf{M}^{-1}\mathbf{C} \end{bmatrix}. \quad (90)$$

For white-noise base acceleration,

$$\mathbf{g} = \begin{bmatrix} \mathbf{0} \\ -\boldsymbol{\tau} \end{bmatrix}, \quad \boxed{\mathbf{B}(t) = 2\pi S_0 \phi^2(t) \mathbf{g} \mathbf{g}^\top}. \quad (91)$$

13.2 Interstorey drift covariance

Define the difference matrix

$$\mathbf{D}_u = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ -1 & 1 & 0 & \cdots & 0 \\ 0 & -1 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & -1 & 1 \end{bmatrix}. \quad (92)$$

Then

$$\mathbf{d}(t) = \mathbf{D}_u \mathbf{u}(t), \quad (93)$$

and the interstorey drift covariance is

$$\boxed{\mathbf{R}_d(t) = \mathbf{D}_u \mathbf{R}_{uu}(t) \mathbf{D}_u^\top}. \quad (94)$$

The storey standard deviations are

$$\sigma_{d_i}(t) = \sqrt{[\mathbf{R}_d(t)]_{ii}}. \quad (95)$$

For drift ratios and storey heights H_i , define

$$\mathbf{D}_{\text{IDR}} = \text{diag}(H_1^{-1}, \dots, H_n^{-1}) \mathbf{D}_u, \quad (96)$$

so that

$$\mathbf{R}_{\text{IDR}}(t) = \mathbf{D}_{\text{IDR}} \mathbf{R}_{uu}(t) \mathbf{D}_{\text{IDR}}^\top. \quad (97)$$

13.3 Acceleration covariance

Relative acceleration is the linear output

$$\ddot{\mathbf{u}} = \mathbf{D}_a \mathbf{z} - \tau a_g, \quad \mathbf{D}_a = \begin{bmatrix} -\mathbf{M}^{-1} \mathbf{K} & -\mathbf{M}^{-1} \mathbf{C} \end{bmatrix}. \quad (98)$$

When the ground-motion filter is included in the augmented state, all terms are linear in the augmented state and

$$\boxed{\mathbf{R}_{\ddot{\mathbf{u}}\ddot{\mathbf{u}}}(t) = \mathbf{D}_a \mathbf{R}(t) \mathbf{D}_a^\top.} \quad (99)$$

For absolute floor acceleration,

$$\ddot{\mathbf{u}}_{\text{abs}} = \ddot{\mathbf{u}} + \tau a_g, \quad (100)$$

and the corresponding output matrix must be assembled consistently with the selected augmented filter realization.

14 Input filters and augmented state space

A colored non-stationary input is generated by appending filter states. This second-order form is the Kanai–Tajimi ground-motion filter, used here in the state-augmented form adopted by Ge et al. [22], whose augmentation preserves the linear structure required by eq. (1). For a second-order filter,

$$\ddot{x}_f + 2\zeta_f \omega_f \dot{x}_f + \omega_f^2 x_f = -\phi(t)w(t), \quad (101)$$

with filter state

$$\mathbf{q}_f = \begin{bmatrix} x_f \\ \dot{x}_f \end{bmatrix}, \quad (102)$$

and

$$\dot{\mathbf{q}}_f = \mathbf{A}_f \mathbf{q}_f + \mathbf{g}_f \phi(t)w(t), \quad (103)$$

where

$$\mathbf{A}_f = \begin{bmatrix} 0 & 1 \\ -\omega_f^2 & -2\zeta_f \omega_f \end{bmatrix}, \quad \mathbf{g}_f = \begin{bmatrix} 0 \\ -1 \end{bmatrix}. \quad (104)$$

For an augmented state

$$\mathbf{z}_a = \begin{bmatrix} \mathbf{u} \\ \dot{\mathbf{u}} \\ \mathbf{q}_f \end{bmatrix}, \quad (105)$$

the same solver is used with an augmented $\mathbf{A}_a(t)$ and

$$\mathbf{B}_a(t) = 2\pi S_0 \phi^2(t) \mathbf{g}_a \mathbf{g}_a^\top. \quad (106)$$

If the filter frequency and damping are constant, $\mathbf{A}_a$ is constant and only $\mathbf{B}_a(t)$ varies. If $\omega_f = \omega_f(t)$ or $\zeta_f = \zeta_f(t)$, then $\mathbf{A}_a = \mathbf{A}_a(t)$.

15 Tolerance selection and numerical practice

15.1 Recommended starting values

Purpose	Suggested initial settings
Exploratory analysis	RelTol=1e-3, with a physically scaled AbsTol.
Routine analysis	RelTol=1e-4 or 1e-5.
Validation	Repeat with tolerances reduced by a factor of 10.
Publication	Provide a work-precision plot or comparison with an exact/high-accuracy reference.

Backward Euler remains first order. Extremely small tolerances may lead to very many steps. For high accuracy, an L-stable higher-order method such as TR-BDF2 may be more efficient.

15.2 Selecting StateScale

For

$$\mathbf{z} = \begin{bmatrix} \mathbf{u} \\ \dot{\mathbf{u}} \\ \mathbf{q}_f \end{bmatrix}, \quad (107)$$

a possible scale vector is

$$\mathbf{s} = \begin{bmatrix} 0.05 \mathbf{1}_u \\ 0.50 \mathbf{1}_v \\ \mathbf{s}_f \end{bmatrix}. \quad (108)$$

The entries are state scales, not variance scales. Covariance entries are automatically scaled by $s_i s_j$.

16 Validation and included tests

The command

```
1 run_tests
```

executes tests for:

1. an exact scalar differential Lyapunov equation;
2. convergence to a stationary algebraic covariance;
3. symmetry and positive semidefiniteness;
4. increasing computational work as tolerances are tightened.

16.1 Exact scalar solution

For

$$\dot{R}(t) = 2aR(t) + b(t), \quad (109)$$

the integrating-factor solution is

$$R(t) = e^{2a(t-t_0)} R_0 + \int_{t_0}^t e^{2a(t-\tau)} b(\tau) d\tau. \quad (110)$$

For constant b and $a \neq 0$,

$$R(t) = -\frac{b}{2a} + \left(R_0 + \frac{b}{2a} \right) e^{2a(t-t_0)}. \quad (111)$$

The stationary value is

$$\bar{R} = -\frac{b}{2a}, \quad a < 0. \quad (112)$$

The backward-Euler scalar update is

$$R_{n+1} = \frac{R_n + h_n b_{n+1}}{1 - 2a_{n+1} h_n}. \quad (113)$$

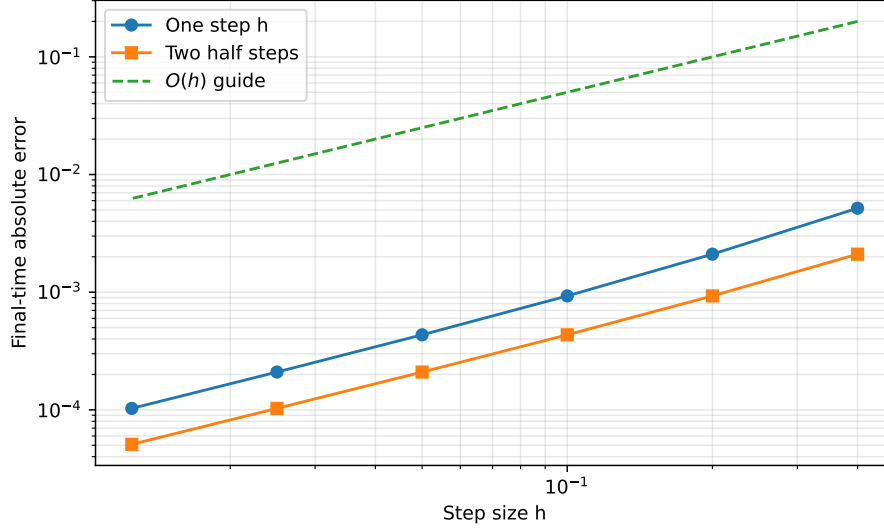


Figure 7: Final-time error illustrating first-order convergence.

16.2 Recommended independent checks

For research use, additionally verify:

- convergence under reductions of `RelTol`, `AbsTol`, and `MaxStep`;
- the stationary residual $\|A\bar{R} + \bar{R}A^\top + B\|_F$;
- agreement with direct quadrature of eq. (12) for a small model;
- agreement with Monte Carlo simulation for at least one benchmark;
- dimensional consistency of S_0 , $\phi(t)$, G , and $B(t)$;
- stability of the augmented state matrix.

17 Extension to first- and second-order sensitivities

Direct differentiation of the covariance equation provides parameter sensitivities without finite differences, in the spirit of nonstationary reliability-sensitivity analysis [23]. Let $\mathbf{b} = [b_1, \dots, b_{n_b}]^\top$ be a design-parameter vector and write

$$\mathbf{R}_{,j} = \frac{\partial \mathbf{R}}{\partial b_j}, \quad \mathbf{R}_{,jk} = \frac{\partial^2 \mathbf{R}}{\partial b_j \partial b_k}. \quad (114)$$

Differentiating eq. (1) gives the first-order equation

$$\dot{\mathbf{R}}_{,j} = \mathbf{A}\mathbf{R}_{,j} + \mathbf{R}_{,j}\mathbf{A}^\top + \mathbf{C}_j(t), \quad (115)$$

where

$$\mathbf{C}_j = \mathbf{A}_{,j}\mathbf{R} + \mathbf{R}\mathbf{A}_{,j}^\top + \mathbf{B}_{,j}. \quad (116)$$

The second-order equation is

$$\dot{\mathbf{R}}_{,jk} = \mathbf{A}\mathbf{R}_{,jk} + \mathbf{R}_{,jk}\mathbf{A}^\top + \mathbf{D}_{jk}(t), \quad (117)$$

with

$$\boxed{D_{jk} = A_{,j}R_{,k} + R_{,k}A_{,j}^\top + A_{,k}R_{,j} + R_{,j}A_{,k}^\top + A_{,jk}R + RA_{,jk}^\top + B_{,jk}}. \quad (118)$$

All equations share the same homogeneous Lyapunov operator.

17.1 Backward-Euler sensitivity updates

At time t_{n+1} , the first-order update satisfies

$$M_{n+1}R_{,j,n+1} + R_{,j,n+1}M_{n+1}^\top + \left(C_{j,n+1} + \frac{1}{h_n}R_{,j,n}\right) = \mathbf{0}. \quad (119)$$

The second-order update is

$$M_{n+1}R_{,jk,n+1} + R_{,jk,n+1}M_{n+1}^\top + \left(D_{jk,n+1} + \frac{1}{h_n}R_{,jk,n}\right) = \mathbf{0}. \quad (120)$$

At each full or half step, the correct cascade is:

1. solve the primal covariance;
2. assemble and solve all first-order sensitivities;
3. assemble and solve all second-order sensitivities.

The Schur factorization of M_{n+1} is shared by every member.

17.2 Dimensionless error control for sensitivities

Use normalized derivatives

$$\tilde{R}_{,j} = b_j R_{,j}, \quad \tilde{R}_{,jk} = b_j b_k R_{,jk}. \quad (121)$$

A family error may be defined as

$$\boxed{\text{err}_{\text{family}} = \max\left(\text{err}_R, \max_j \text{err}_{b_j R_{,j}}, \max_{j \leq k} \text{err}_{b_j b_k R_{,jk}}\right)}. \quad (122)$$

17.3 Output functionals

For a scalar quadratic response measure

$$g(\mathbf{b}, t) = \mathbf{y}^\top R(t, \mathbf{b}) \mathbf{y}, \quad (123)$$

with parameter-independent $\mathbf{y}$,

$$g_{,j} = \mathbf{y}^\top R_{,j} \mathbf{y}, \quad g_{,jk} = \mathbf{y}^\top R_{,jk} \mathbf{y}. \quad (124)$$

If $\mathbf{y} = \mathbf{y}(\mathbf{b})$, product-rule terms must also be included.

18 Second-order uncertainty propagation

Let

$$\mathbf{b} = \bar{\mathbf{b}} + \delta\mathbf{b}, \quad \mathbb{E}[\delta\mathbf{b}] = \mathbf{0}, \quad \mathbb{E}[\delta\mathbf{b} \delta\mathbf{b}^\top] = \mathbf{\Sigma}_b. \quad (125)$$

For a scalar response $g(\mathbf{b})$, the second-order Taylor expansion is

$$g(\mathbf{b}) \approx g(\bar{\mathbf{b}}) + \mathbf{g}_b^\top \delta\mathbf{b} + \frac{1}{2} \delta\mathbf{b}^\top \mathbf{H}_g \delta\mathbf{b}. \quad (126)$$

The second-order mean approximation is

$$\mathbb{E}[g] \approx g(\bar{\mathbf{b}}) + \frac{1}{2} \text{tr}(\mathbf{H}_g \mathbf{\Sigma}_b). \quad (127)$$

For Gaussian parameter uncertainty, the variance approximation is

$$\text{Var}[g] \approx \mathbf{g}_b^\top \mathbf{\Sigma}_b \mathbf{g}_b + \frac{1}{2} \text{tr}(\mathbf{H}_g \mathbf{\Sigma}_b \mathbf{H}_g \mathbf{\Sigma}_b). \quad (128)$$

These formulas explain the value of the covariance Hessian in robust design and uncertainty propagation.

19 Method limitations

- The method has global order one. Alternative non-stationary response-determination methods are available [24, 25], and higher-order L-stable integration is treated in the companion document, at the cost of the unconditional positive-semidefiniteness guarantee retained here.
- Step doubling requires three Lyapunov solves per attempt.
- Full covariance-history storage may dominate memory for large systems.
- PSD projection is a diagnostic repair, not a substitute for model verification.
- A non-PSD $\mathbf{B}(t)$ generally destroys the covariance interpretation.
- The solver is for linear or prescribed linear time-varying systems. If $\mathbf{A}$ depends on $\mathbf{R}$, an inner nonlinear iteration is required, as in equivalent stochastic linearization [2].
- Dense Schur methods are not optimal for very large sparse or low-rank problems; Krylov or low-rank methods may be preferable.

20 Troubleshooting

Symptom	Recommended check
Very many small steps	Check <code>StateScale</code> , discontinuities in $\mathbf{A}(t)$ or $\mathbf{B}(t)$, and whether tolerances are realistic for a first-order method.
Failure at <code>MinStep</code>	Reduce <code>MinStep</code> , relax tolerances, or use a higher-order method.
Significant negative eigenvalues	Verify $\mathbf{B}(t) \succeq 0$, stability, units, and algebraic residuals before enabling PSD projection.

Symptom	Recommended check
Nearly singular Lyapunov operator	Check condition eq. (25) and the shifted spectrum.
Excessive memory	Store selected functionals through <code>OutputFcn</code> , decimate output, or modify the code for streaming mode.
Different result from MATLAB <code>lyap</code>	Confirm the sign convention: this package solves $AX + XA^T + Q = 0$.
Unexpectedly slow constant-A case	Check whether step sizes change at every interval, preventing factorization-cache reuse.

A Complete pseudocode

```

1  t = t0
2  R = R0
3  h = h0
4  previous_error = 1
5
6  while t < tf
7      h = min(h, tf-t)
8
9      % One full backward-Euler Lyapunov step
10     Rfull = BE_Lyapunov(R, t, h)
11
12     % Two consecutive half steps
13     Rhalf1 = BE_Lyapunov(R, t, h/2)
14     Rhalf2 = BE_Lyapunov(Rhalf1, t+h/2, h/2)
15
16     % Step-doubling error
17     E = Rhalf2 - Rfull
18     err = scaled_error(E, Rhalf2, R)
19
20     if err <= 1
21         t = t + h
22         R = 0.5*(Rhalf2 + Rhalf2')
23         check_or_project_PSD(R)
24         store_output(t,R,h,err)
25         h = PI_or_P_update(h,err,previous_error)
26         previous_error = max(err,realmin)
27     else
28         h = rejected_step_update(h,err)
29         % t and R remain unchanged
30     end
31 end

```

The elementary substep is

```

1  function Rnew = BE_Lyapunov(Rold,t,h)
2      Aend = A(t+h)
3      Bend = B(t+h)
4
5      M = Aend - I/(2*h)
6      Q = Bend + Rold/h
7
8      solve M*Rnew + Rnew*M' + Q = 0
9      Rnew = 0.5*(Rnew + Rnew')
10 end

```

B Output callback

```

1  opts.OutputFcn = @my_output;
2  sol = lyap_be_adaptive(A,B,[0 30],R0,opts);
3
4  function stop = my_output(t,R,flag)
5      persistent fid
6      stop = false;
7      switch flag
8          case 'init'
9              fid = fopen('variance_history.txt','w');
10             case 'step'
11                 fprintf(fid,'% .12e % .12e\n',t,R(1,1));
12             case 'done'
13                 fclose(fid);
14         end
15     end

```

Version 1.0 still stores `sol.R`. For very large problems, a future streaming mode should explicitly disable full-history storage.

C Reproducibility checklist

For published numerical results, report at least:

- MATLAB version and operating system;
- processor, RAM, and number of numerical-library threads;
- `RelTol`, `AbsTol`, `InitialStep`, `MinStep`, and `MaxStep`;
- `StateScale`;
- accepted and rejected steps;
- number of Lyapunov solves and Schur factorizations;
- maximum algebraic residual;
- PSD diagnostics;
- independent validation criterion.

References

- [1] R. H. Bartels and G. W. Stewart. Solution of the matrix equation $AX + XB = C$. *Communications of the ACM*, 15(9):820–826, 1972. doi: 10.1145/361573.361582.
- [2] G. Acciani, F. Di Modugno, A. Abrescia, and G. C. Marano. Integration algorithm for covariance nonstationary dynamic analysis using equivalent stochastic linearization. *Mathematics and Computers in Simulation*, 125:70–82, 2016. doi: 10.1016/j.matcom.2015.11.006.
- [3] M. Domaneschi, R. Cucuzza, L. Sardone, S. Londoño Lopez, M. Movahedi, and G. C. Marano. Numerical covariance evaluation for linear structures subject to non-stationary random inputs. *Computation*, 12(3):50, 2024. doi: 10.3390/computation12030050.
- [4] G. C. Marano, G. Acciani, and L. G. Cascella. Non-stationary numerical covariance analysis of linear multi-degree-of-freedom mechanical systems subject to random inputs. *International Journal of Computational Methods*, 4(1):173–194, 2007. doi: 10.1142/S0219876207001072.
- [5] G. C. Marano, G. Acciani, and A. Abrescia. Numerical algorithm for non-stationary covariance analysis of nonlinear mechanical systems using equivalent stochastic linearization. In *Proceedings of the ASME 2014 12th Biennial Conference on Engineering Systems Design and Analysis (ESDA2014), Volume 2: Dynamics, Vibration and Control; Energy; Fluids Engineering; Micro and Nano Manufacturing*, Copenhagen, Denmark, 2014. doi: 10.1115/ESDA2014-20448.
- [6] G. C. Marano, G. Acciani, A. Fiore, and A. Abrescia. Integration algorithm for covariance nonstationary dynamic analysis of SDOF systems using equivalent stochastic linearization. *International Journal of Structural Stability and Dynamics*, 15(2):1450044, 2015. doi: 10.1142/S0219455414500448.
- [7] A. Fiore, L. Sardone, R. Greco, R. Cucuzza, and G. C. Marano. Numerical approach for non-stationary covariance analysis in linear random vibrations applied to a multi-storey building under seismic action. *International Journal of Structural Stability and Dynamics*, 25(13):2650198, 2025. doi: 10.1142/S0219455426501981.
- [8] G. Muscolino and T. Alderucci. Closed-form solutions for the evolutionary frequency response function of linear systems subjected to separable or non-separable non-stationary stochastic excitations. *Probabilistic Engineering Mechanics*, 40:75–89, 2015.
- [9] S. Caddemi, P. Colajanni, I. Duca, and G. Muscolino. Non-geometric spectral moments for frequency-varying filtered input processes. *Probabilistic Engineering Mechanics*, 19(1–2):21–31, 2004. doi: 10.1016/j.probengmech.2003.11.003.
- [10] J. Luo, G. K. Er, V. P. Iu, and C. C. Lam. Transient PDF solution of nonlinear stochastic oscillator subjected to modulated Gaussian white noise. *International Journal of Structural Stability and Dynamics*, 4(1):2450011, 2024.
- [11] L. D. Lutes and S. Sarkani. *Random Vibrations: Analysis of Structural and Mechanical Systems*. Elsevier Butterworth-Heinemann, Oxford, 2004.
- [12] H. A. Buchholdt and E. Moossavi Nejad. *Structural Dynamics for Engineers*. ICE Publishing, 2011.
- [13] C. F. Van Loan. Computing integrals involving the matrix exponential. *IEEE Transactions on Automatic Control*, 23(3):395–404, 1978. doi: 10.1109/TAC.1978.1101743.
- [14] E. Hairer and G. Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. Springer, Berlin, 2 edition, 1996. doi: 10.1007/978-3-642-05221-7.

- [15] C. Choi. A variable step-size method for solving stiff Lyapunov differential equations. In *Proceedings of the American Control Conference*, pages 4197–4199, 2001.
- [16] C. Choi. Improving the accuracy of a variable step-size method for solving stiff Lyapunov differential equations. In *Proceedings of the American Control Conference*, 2003.
- [17] P. C. Jennings, G. W. Housner, and N. C. Tsai. Simulated earthquake motions. Technical report, California Institute of Technology, Pasadena, CA, 1968.
- [18] M. Shinozuka and Y. Sato. Simulation of nonstationary random process. *Journal of the Engineering Mechanics Division*, 93(1):11–40, 1967.
- [19] T. Hsu and M. Bernard. A random process for earthquake simulation. *Earthquake Engineering & Structural Dynamics*, 6(4):347–362, 1978.
- [20] G. C. Marano. Non-stationary stochastic modulation function definition based on process energy release. *Physica A: Statistical Mechanics and its Applications*, 517:280–289, 2019.
- [21] R. W. Clough and J. Penzien. *Dynamics of Structures*. McGraw-Hill, New York, 1975.
- [22] X. Ge, C. Li, I. Azim, J. Gong, and Y. Li. Structural dynamic responses of linear structures subjected to Kanai–Tajimi excitation. *Structures*, 34:3958–3967, 2021.
- [23] R. Greco and F. Trentadue. Structural reliability sensitivities under nonstationary random vibrations. *Mathematical Problems in Engineering*, 2013:1–21, 2013. doi: 10.1155/2013/426361.
- [24] F. Kong, R. Han, S. Li, and W. He. Non-stationary approximate response of nonlinear multi-degree-of-freedom systems subjected to combined periodic and stochastic excitation. *Mechanical Systems and Signal Processing*, 166:108420, 2022.
- [25] P. D. Spanos and W. Zhang. Nonstationary stochastic response determination of nonlinear oscillators endowed with fractional derivatives. *International Journal of Non-Linear Mechanics*, 146:104170, 2022.